

Parallel computing for Deep Learning

Content

- ~~■ benefits of parallelization~~
- parallelization strategies
- ~~■ Hands on: hyperparameter grid search on an HPC system~~
- ~~■ parallel stochastic gradient descent (SGD)~~
- ~~■ synchronous parallel SGD~~
- ~~■ communication backends~~
- frameworks for distributed deep learning
- documentation of distributed DL frameworks

Types of parallelization

What types of parallelization exist?

- Instruction level parallelism
- Embarrassingly parallel
- Data parallel
- Model parallel:
 - Pipeline parallel
 - Tensor parallel
 - Sequence parallel
- Hybrid data/model parallelism



Increasing complexity/bandwidth

Types of parallelization

- Instruction level parallelism
 - Executing multiple instructions (e.g. additions) in parallel
 - Examples: vector instructions (CPU), Tensor Cores (GPU)
 - https://en.wikipedia.org/wiki/Instruction-level_parallelism

$$\begin{array}{rcccl} X = & \boxed{1.1} & \boxed{3.7} & \boxed{-1.6} & \boxed{2.3} \\ & & + & & \\ Y = & \boxed{-3.4} & \boxed{1.7} & \boxed{-0.2} & \boxed{5.2} \\ & & = & & \\ & & \boxed{-2.3} & \boxed{5.4} & \boxed{-1.8} & \boxed{7.5} \end{array}$$

Types of parallelization

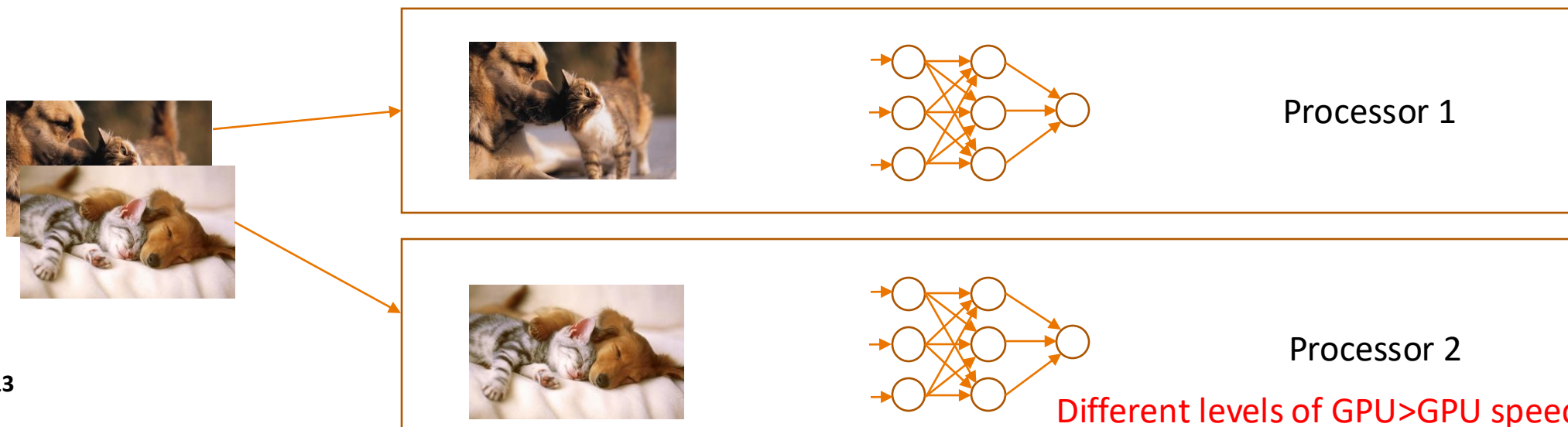
- Embarrassingly parallel
 - A workload or problem where little or no effort is needed to separate into number of parallel tasks
 - Examples: hyperparameter grid search, training multiple model architectures
 - https://en.wikipedia.org/wiki/Embarrassingly_parallel

Data Parallelism

Train a single model, single set of hyperparameters, but **faster**

$$\left(\frac{N}{n_{gpus}}, S, D\right)$$

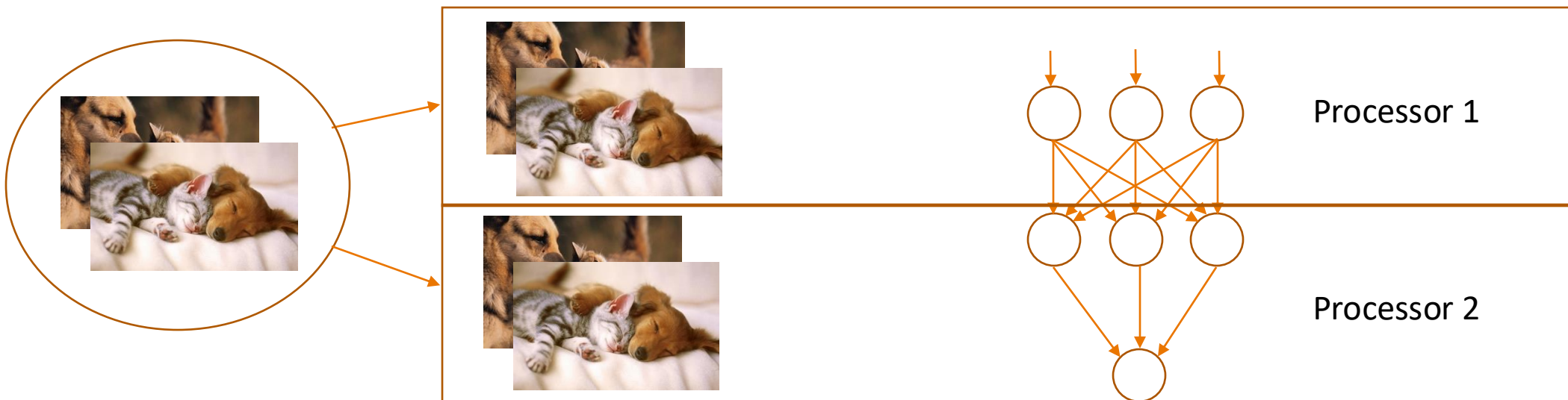
- Split the batch over multiple processors (CPUs/GPUs)
- Each processor holds a copy of the model
- Forward pass: calculated by each of the workers
- Backward pass: gradients computed (per worker), communicated and aggregated
- Effective batch size is a multiple of the number of workers



Model parallelism

Train a single **very big** model, single set of hyperparameters

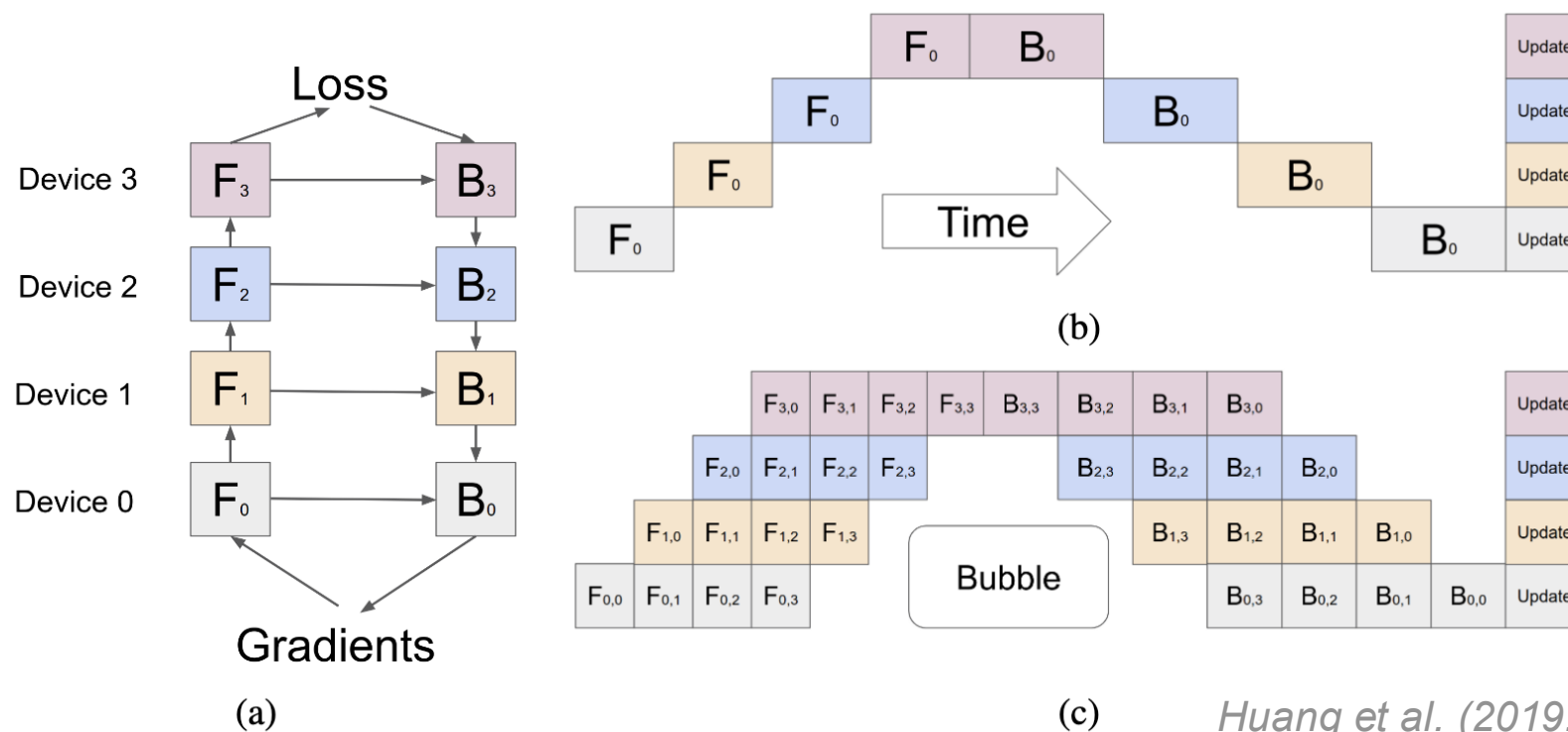
- Split the model over multiple processors (CPUs/GPUs or a hybrid)
- Each processor sees all the data
- Communication needed both during forward and backward pass!



Model parallelism: Pipeline parallelism

- Model parallelism, executed in a pipelined fashion over multiple micro-batches (data parallelism).
- More efficient than model parallelism: hides communication with computation
- See <https://pytorch.org/docs/stable/distributed.pipelining.html>

$$\left(N, S, \frac{D}{n_{gpus}}\right)$$



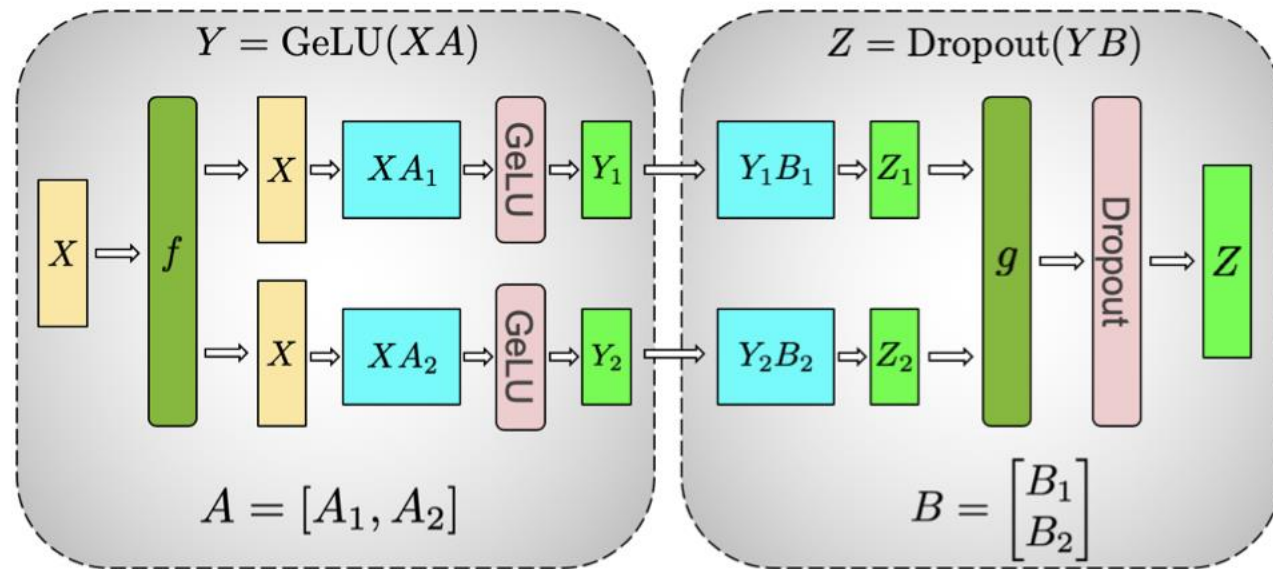
Model parallelism: Tensor parallelism

Split individual weight W and/or feature X tensors across devices:

- $Y = \text{ReLU}(X W)$
- $X = [X_1, X_2], W = \begin{bmatrix} W_1 \\ W_2 \end{bmatrix} \rightarrow Y = \text{ReLU}(X_1 W_1 + X_2 W_2)$
- Requires device sync due to non-linearity a synchronization is required, instead split $W = [W_1, W_2]$
- $[Y_1, Y_2] = [\text{ReLU}(XW_1), \text{ReLU}(XW_2)]$

$$\left(N, S, \frac{D}{n_{\text{gpus}}}\right)$$

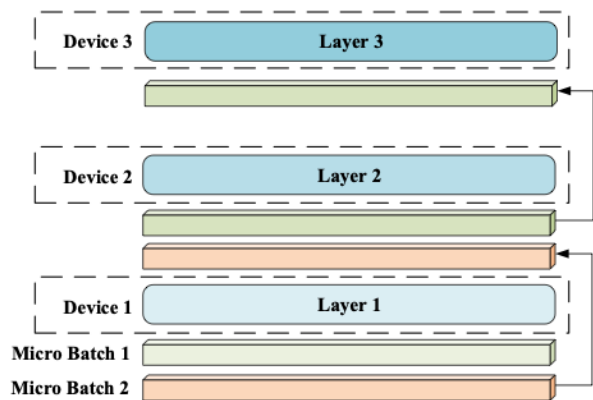
<https://arxiv.org/pdf/1909.08053>



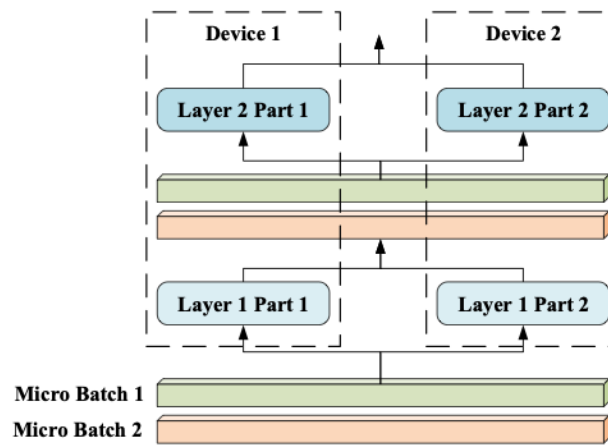
Model parallelism: Sequence parallel

- Split longer sequences (e.g. sentences) across devices.
- Higher maximum batch size and throughput compared to tensor parallel

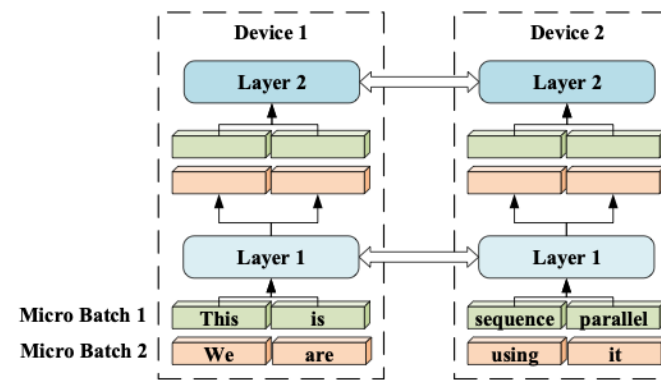
$$\left(N, \frac{S}{n_{gpus}}, D\right)$$



(a) Pipeline parallelism

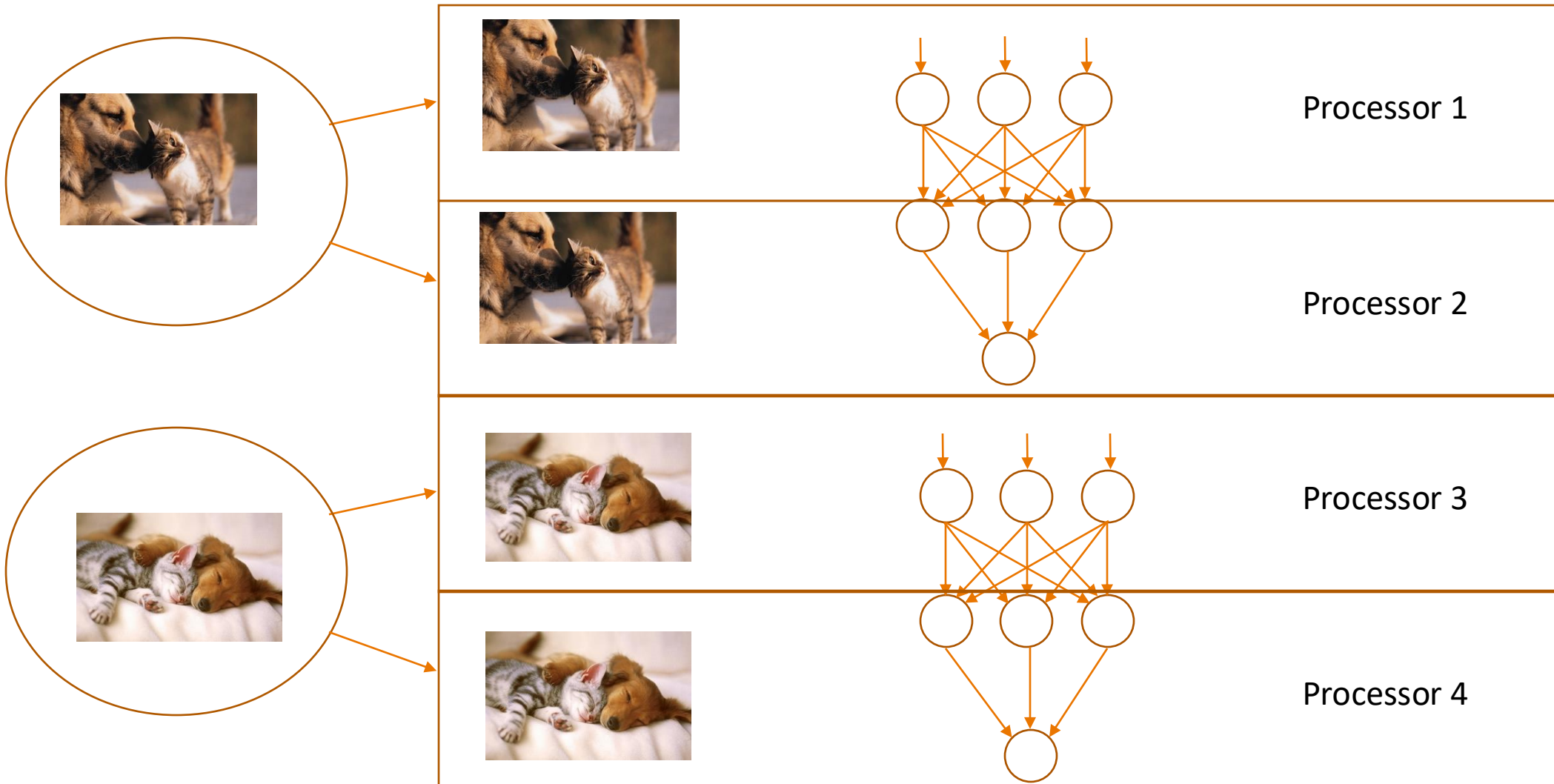


(b) Tensor parallelism



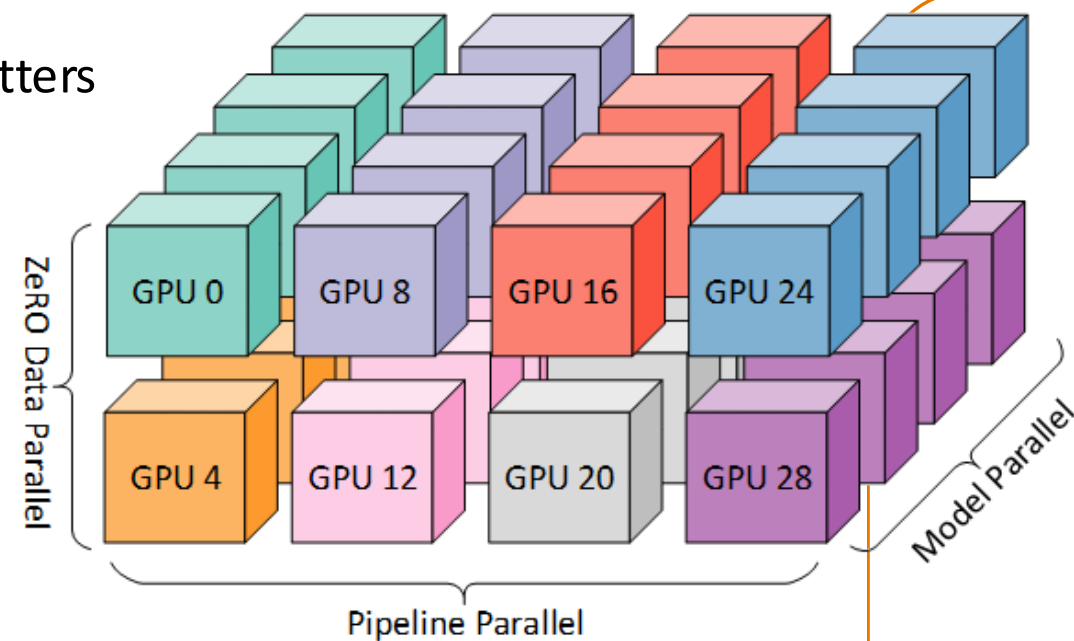
(c) Sequence parallelism (Ours)

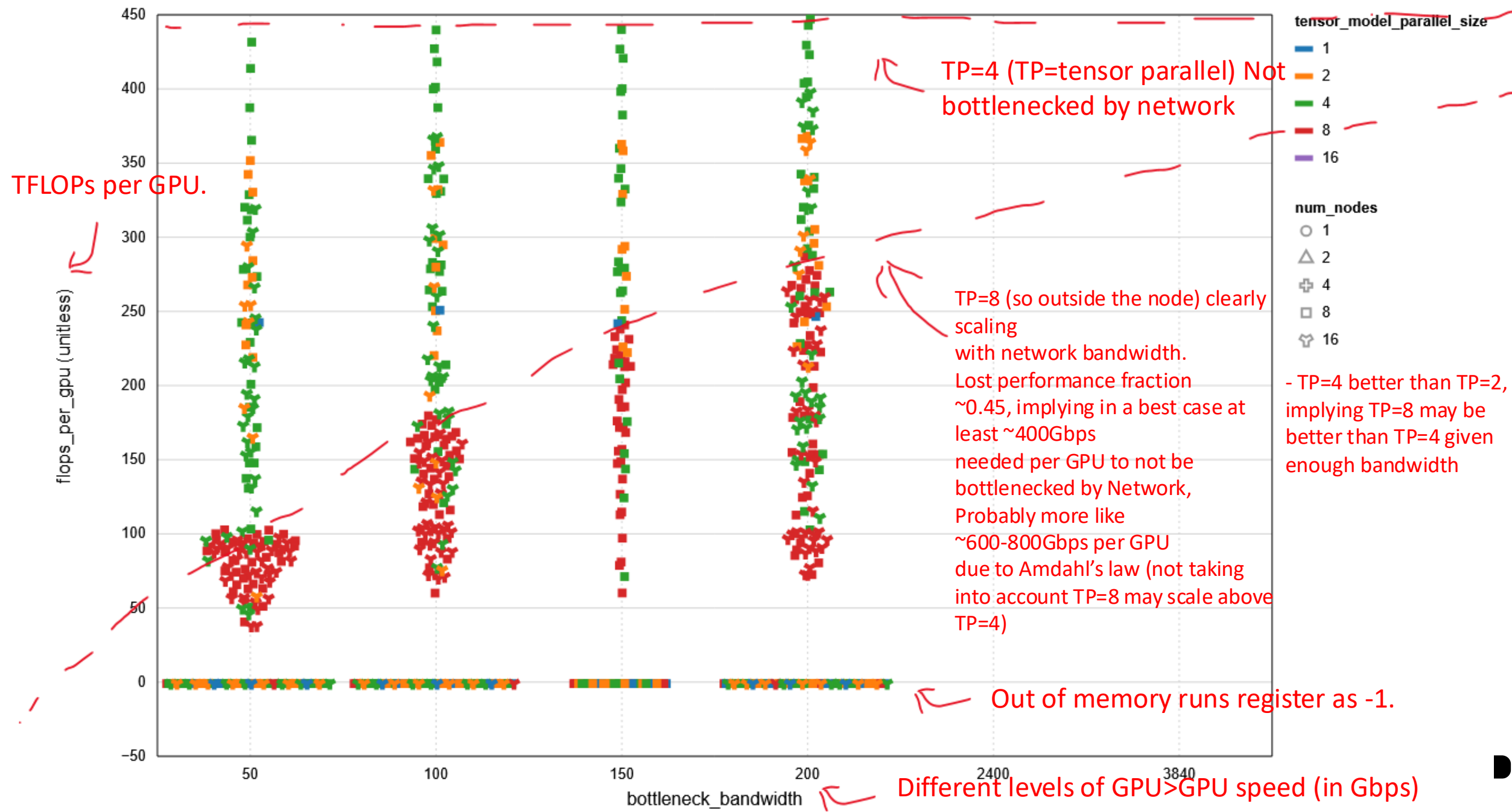
Hybrid model/data parallelism



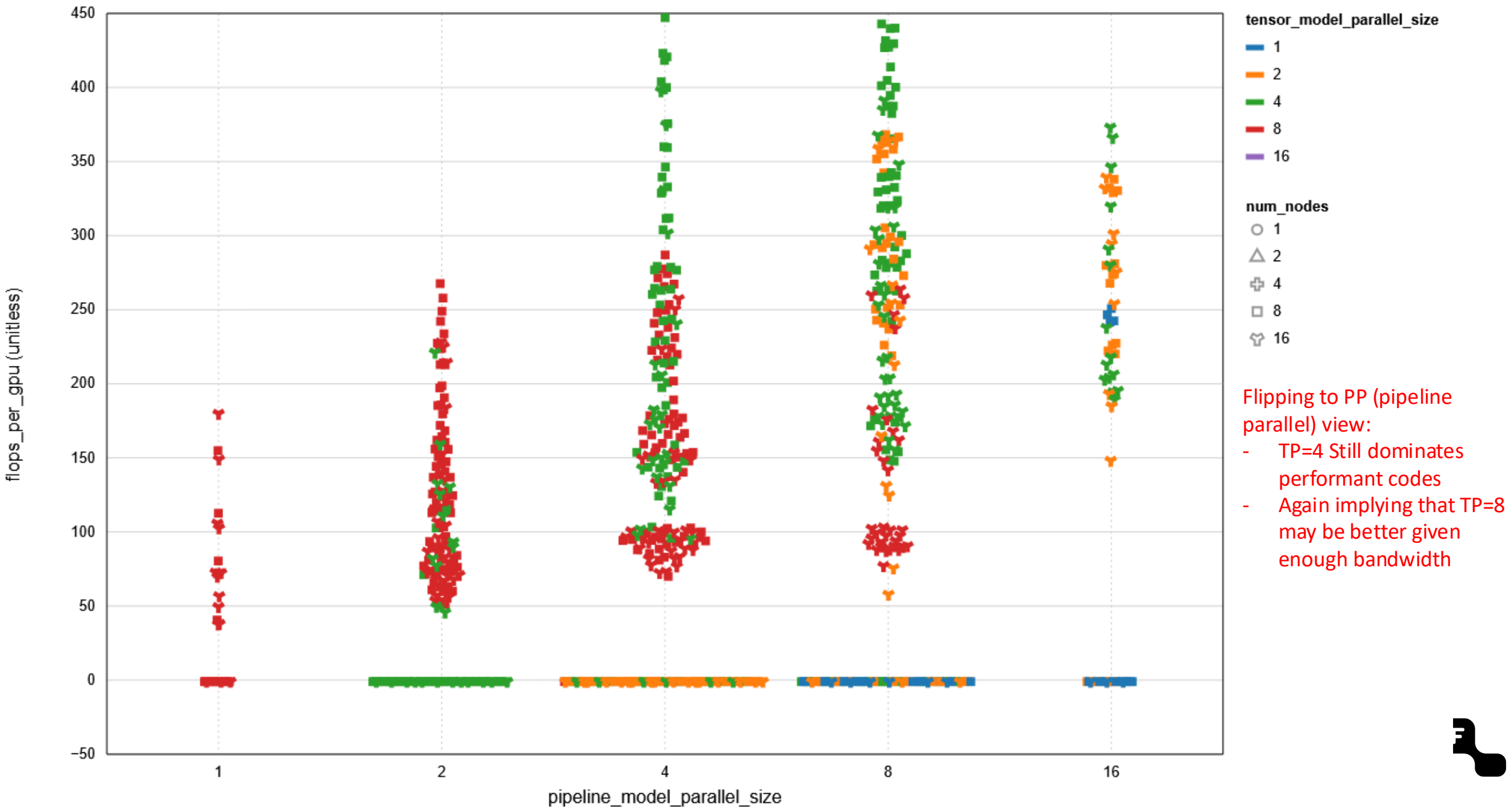
3D Parallelism

- Combined Data Parallel, Pipeline Parallel, Tensor Parallel
 - 4D: including Sequence parallel
- 3D grid of devices
 - Each dimension deals with its own parallelization type
- Physical location of GPUs with respect to each other matters
- For example:
 - Tensor parallel between GPUs in node
 - Pipeline parallel between nodes
 - Data parallel between racks

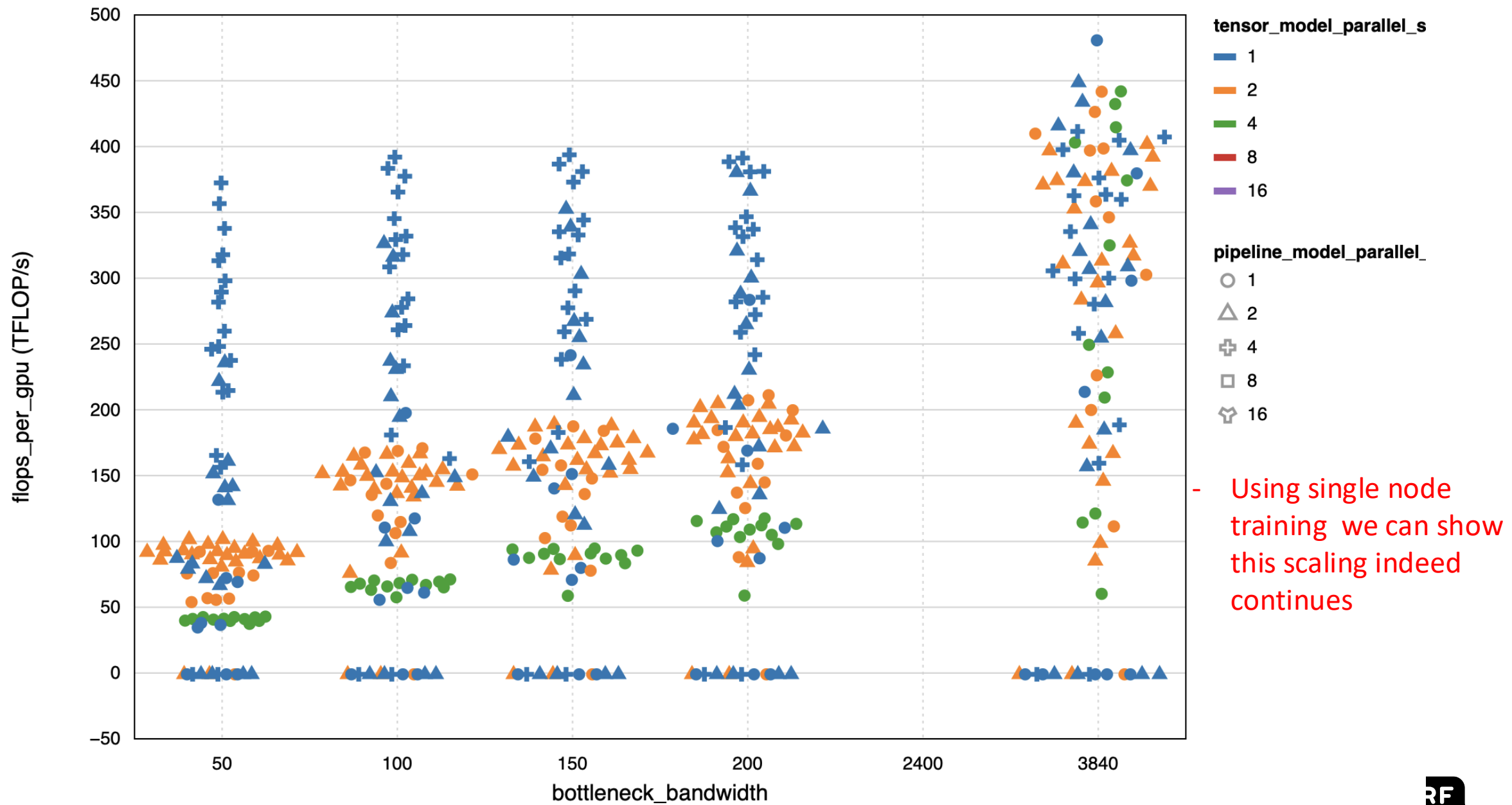




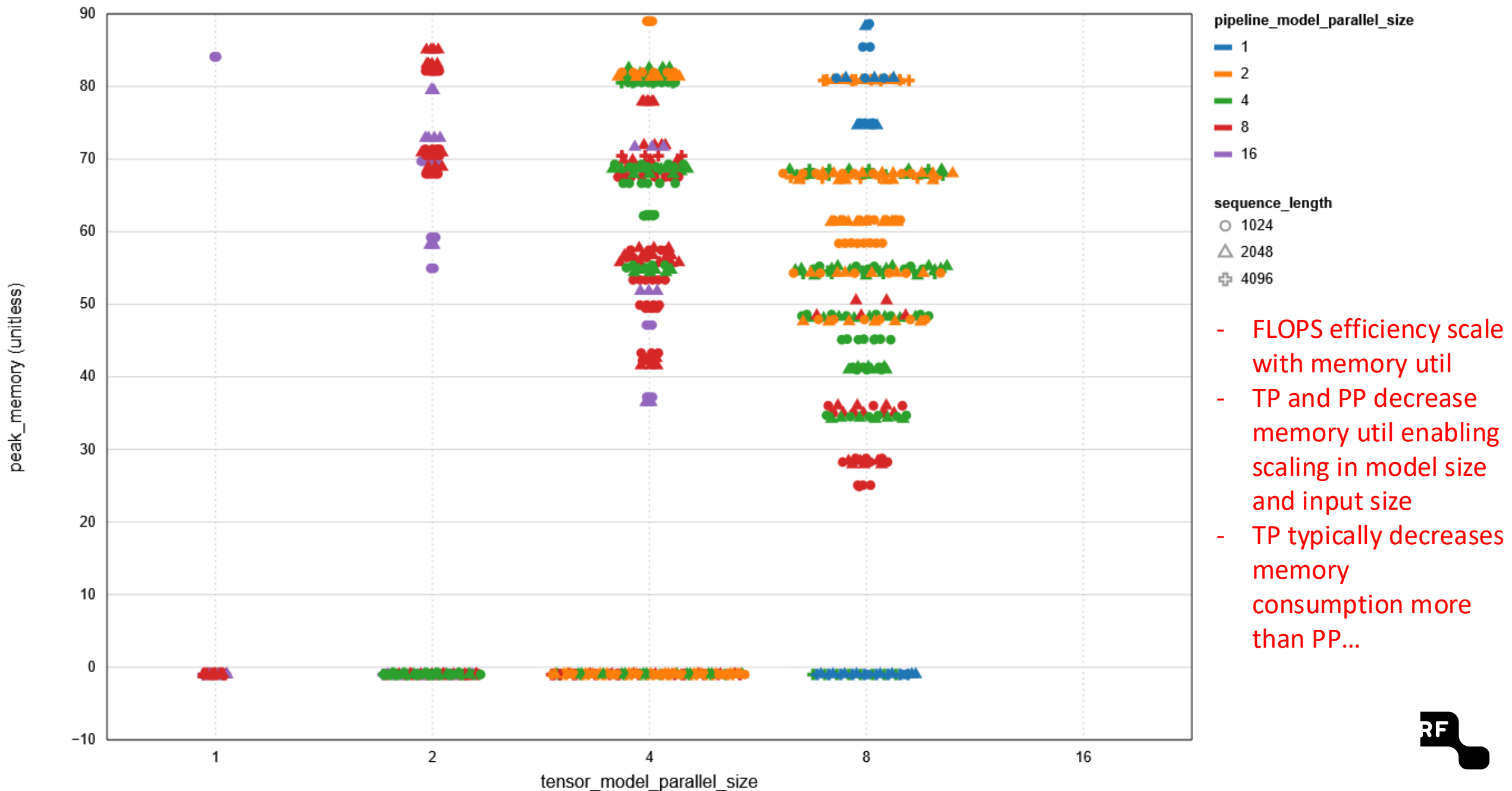
MegatronTrainTest / default (study "PodsOrNot")
gpus_per_node=4 • model_size_billions=70 • num_query_groups=<missing>



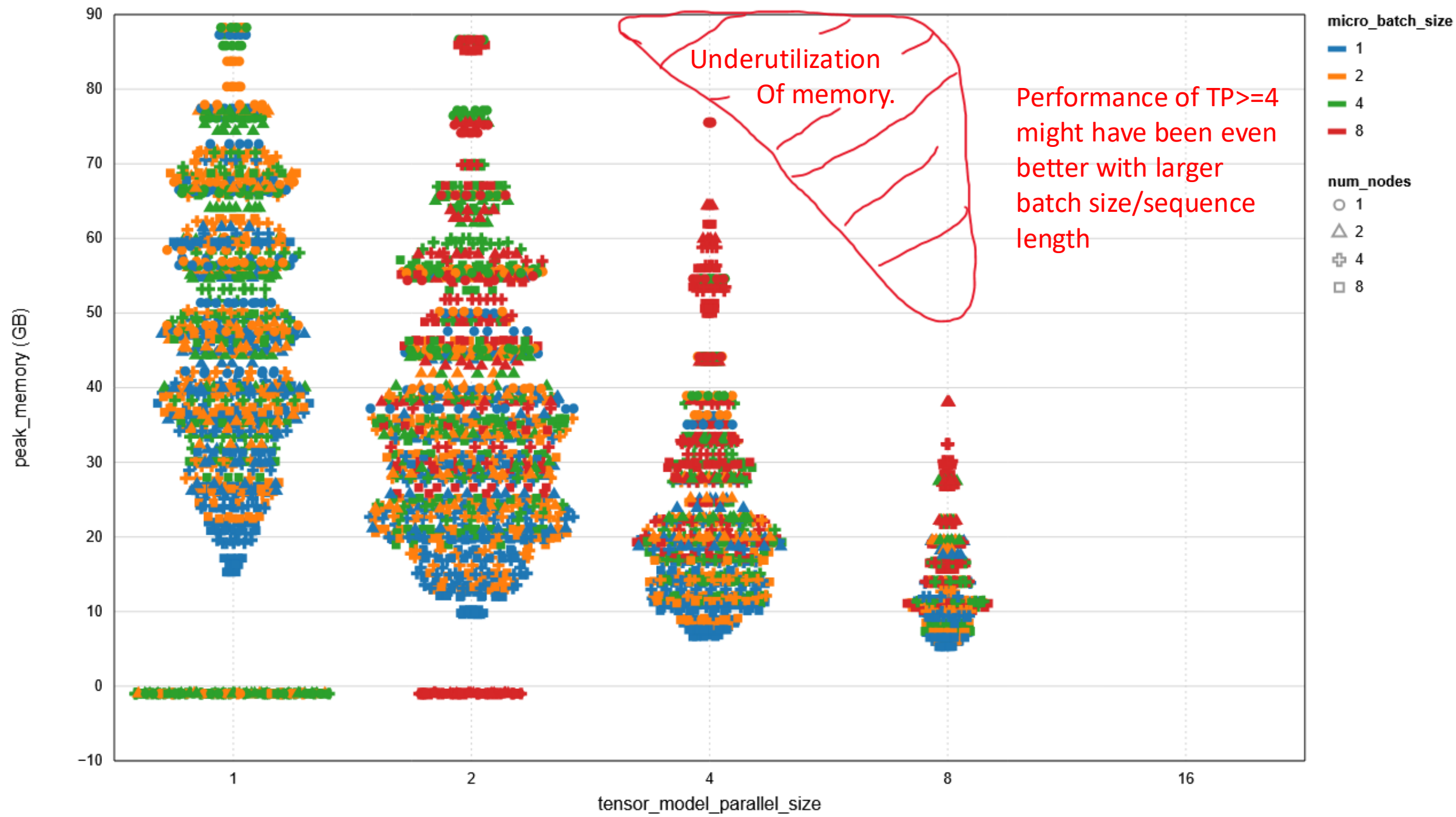
Megatron Train test / default (study - PodsOrNot)
gpus_per_node=4 • model_size_billions=8 • num_nodes=1



MegatronTrainTest / default (study "PodsOrNot")
 gpus_per_node=4 • model_size_billions=70 • num_query_groups=<missing>



Megatron Train Test / default (study "PodsOrNot")
gpu_per_node=4 • model_size_billions=8 • num_query_groups=<missing>



What type of parallelism is applicable? Ease-of-implementation

- Multiple (independent) training runs => Embarrassingly parallel
- Single model takes too long to train => Data parallel
- Single model is too big for memory => Model / Pipeline / Tensor / Sequence
- Single model is too big for memory *and* takes too long to train => Hybrid parallelism
 - All of the well-known, big models (GPT-X) are trained this way

What type of parallelism is applicable? Ease-of-implementation

- Multiple (independent) training runs => Embarrassingly parallel
- Single model takes too long to train => Data parallel
- Single model is too big for memory => Model / Pipeline / Tensor / Sequence
- Single model is too big for memory *and* takes too long to train => Hybrid parallelism
 - All of the well-known, big models (GPT-X) are trained this way

Any cluster will do, no fast network needed

What type of parallelism is applicable? Ease-of-implementation

- Multiple (independent) training runs => Embarrassingly parallel
- Single model takes too long too train => Data parallel
- Single model is too big for memory => Model / Pipeline / Tensor / Sequence
- Single model is too big for memory *and* takes too long to train => Hybrid parallelism
 - All of the well-known, big models (GPT-X) are trained this way



HPC cluster needed, i.e. with fast network and fast connections between e.g. GPUs in a single node

So..... What type of parallelization? Optimal performance

- Data parallel (DP)
 - Easy to set-up.
 - Relatively infrequent and low-bandwidth communication (only once per forward/backwards)
 - Model is copied to each device, so redundancy in memory that might hamper efficiency
 - FSDP (fully sharded DP) partially solves this by sharding the model and retrieving the weights for a layer just-in-time. Requires more communication.
- Pipeline parallel
 - Complicated to set-up, lots of tuning to avoid bubbles
 - Moderate communication (after each block of layers)
 - Significant memory reduction by splitting model and batches over devices
- Tensor parallel
 - More involved than DP to set-up but does not suffer from bubbles. Computationally very efficient.
 - Very high bandwidth requirement
 - Significant memory reduction. Typically even more than pipeline

Do I need to parallelise?

Consider alternatives:

- Model pruning
- Use different hardware architecture
- Reduced precision datatypes
- Low rank fine-tuning (LoRA/QLoRA)
- Quantization (inference)

Parallelization tools/wrappers

- torch.distributed: <https://pytorch.org/docs/stable/distributed.html>
- tf.distribute: https://www.tensorflow.org/guide/distributed_training
- Data parallel:
 - PyTorch Distributed Data Parallel (DDP)
 - **ZeRO-1:** Partition optimizer state
- Model parallel:
 - DeepSpeed ZeRO
 - **ZeRO-2:** Partition gradients
 - Torch FullyShardedDataParallel (FSDP): <https://pytorch.org/docs/stable/fsdp.html>
 - **ZeRO-3:** Partition Model Weights
 - Wrapper of ZeRO-3
- Huggingface's accelerate: <https://huggingface.co/docs/transformers/main/en/accelerate>

Frameworks for distributed learning

- TensorFlow's tf.distribute: quite tricky to program. Lot's of code changes needed from serial to distributed (https://www.tensorflow.org/guide/distributed_training)
- TensorFlow + Horovod: serial => distributed with minimal code changes (<https://horovod.readthedocs.io/en/stable/tensorflow.html>)
- PyTorch's torch.distributed (https://pytorch.org/tutorials/intermediate/dist_tuto.html)
- PyTorch + Horovod: serial => distributed with minimal code changes (<https://horovod.readthedocs.io/en/stable/pytorch.html>)
- PyTorch Lightning: hides a lot of boiler plate code (also nice for serial training). Very little changes needed between serial & parallel execution, especially on a SLURM cluster (<https://pytorch-lightning.readthedocs.io/en/latest/clouds/cluster.html#slurm-managed-cluster>)

Further reading

- Distributed TensorFlow using Horovod: <https://towardsdatascience.com/distributed-tensorflow-using-horovod-6d572f8790c4>
- Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis: <https://arxiv.org/pdf/1802.09941.pdf>
- Prace best practice guide for Deep Learning: <http://www.prace-ri.eu/IMG/pdf/Best-Practice-Guide-Deep-Learning.pdf>
- Technologies behind Distributed Deep Learning: <https://preferredresearch.jp/2018/07/10/technologies-behind-distributed-deep-learning-allreduce/>
- PyTorch Distributed: https://pytorch.org/tutorials/beginner/dist_overview.html and <https://pytorch.org/docs/stable/distributed.html>